

2026 年英特尔杯大学生电子设计竞赛嵌入式系统专题邀请赛

2026 Intel Cup Undergraduate Electronic Design Contest

- Embedded System Design Invitational Contest

结项项目报告

(Final Project Report)

项目题目： **LabGuardian — 基于边缘 AI 的智能电路实验助教系统**

(A Neuro-Symbolic Edge AI Tutor for Hands-on Analog Circuit Experiments)

学生姓名：

指导教师：

参赛学校：

提交日期：

LabGuardian — 基于边缘 AI 的智能电路实验助教系统

摘要

本报告系统总结 LabGuardian 项目在 2026 年英特尔杯嵌入式系统专题邀请赛期间完成的全部工作。LabGuardian 是一套面向高校电子类基础实验（电路分析、模拟电子技术）的边缘 AI 智能助教系统，以英特尔指定平台 DK-2500 (Intel Core Ultra 5 225U) 为算力中枢，覆盖“视觉感知 → 拓扑重构 → 知识检索 → 智能诊断 → 教学解释”的全流程教学闭环。

在视觉感知层，系统使用 YOLO-Pose 关键点检测结合单应性映射与 Snap-to-Grid 网格吸附算法，解决了密集面包板场景下杜邦线遮挡导致的引脚定位难题；在拓扑重构层，深度优先搜索 (DFS) 图论算法把物理坐标转化为工业标准 SPICE 电气网表，输出 component / port / net 三类节点构成的端口级异构图 HeteroCircuitGraph；在拓扑分类层，本报告提出 CADx (Canonical-Anchored Diagnosis) 神经-符号架构，让数据驱动的图神经网络 GNN-A 与符号化的 Template Matcher 独立判断拓扑类别，再以“共识门”(Consensus Gate) 把二者的一致程度编码为 4 级 confidence_band，为前端“AI 已识别 / 需用户确认 / 双假设并列 / 未知”的差异化交互提供机器可读依据。

在知识检索层，系统建立面向 6 个 demo 拓扑的本地多模态知识库 (2 份 datasheet + 6 个 teaching_scene + 17 个 fault_case)，并引入 Push-Based Context Management (PCM) 把错误家族 → 工具白名单的映射显式编码，防止 LLM 在 ReAct 循环中调用与当前错误无关的工具；同时设计 Anti-Hallucination Routing，对“NE555 的引脚怎么排”这类纯查询性问题绕开 LLM 直接从 KB 渲染答案，把幻觉风险压到零。在编排层，统一的 LangGraph 服务 4 种意图 (diagnostic / concept_tutor / lab_guidance / mixed)，所有意图共享同一组节点，差异仅由 intent 字段在 3 个分支点局部决定。

系统在 6 类标准模拟电路 demo 上完成端到端验证；后端 928 个 pytest 用例通过，0 新增回归。本地 Mac 上 Gemma3-4B 推理 18 s/次，规划在 DK-2500 NPU INT4 量化后降至 2–5 s/次。系统全部推理与知识检索在 DK-2500 本地完成，无外网依赖，满足教学场景的隐私与弱网约束。

关键词：边缘 AI，神经-符号架构，图神经网络，拓扑分类，Push-Based Context Management，反幻觉检索，ReAct 多意图统一，DK-2500 异构计算

目 录

第一部分 项目概述	5
1.1 问题背景与教学痛点	5
1.2 系统目标与设计原则	5
1.3 演示场景与覆盖范围	5
1.4 报告结构	6
第二部分 系统总体架构	7
2.1 CADx 五层神经-符号架构	7
2.1.1 L1 表征层 (Representation Layer)	7
2.1.2 L2 决策层 (Decision Layer) — 共识门	7
2.1.3 L3 检索层 (Retrieval Layer)	8
2.1.4 L4 编排层 (Orchestration Layer)	8
2.1.5 L5 表达层 (Expression Layer)	8
2.2 数据流总览	8
第三部分 视觉感知与拓扑重构	10
3.1 视觉管线概览	10
3.2 关键点检测替代端到端目标检测	10
3.3 Snap-to-Grid 网格吸附	10
3.4 端口级异构电路图 HCG	11
3.5 视觉管线已知限制与缓解	11
第四部分 CADx — 拓扑分类与共识门	12
4.1 GNN-A 拓扑分类器	12
4.1.1 设计动机	12
4.1.2 模型架构	12
4.1.3 数据集生成	12
4.1.4 UA741 三兄弟区分 (v1 → v2 升级)	12
4.2 Topology Template — 参数化不变量框架	13
4.2.1 五字段语义分解	13
4.2.2 Coverage-Aware Subgraph Isomorphism	13
4.3 Consensus Gate — 共识门	14
4.3.1 决策规则	14
4.3.2 教学价值	14
4.4 Wire-Aware Role Propagation	14
4.4.1 设计观察	15
4.4.2 算法机制	15
4.4.3 实证效果	15
第五部分 智能 Agent 与多模态 RAG	16
5.1 Intent-Unified ReAct 编排	16
5.1.1 4 种意图统一在同一图中	16

5.1.2 LLM-Hybrid 意图分类	16
5.2 Push-Based Context Management (PCM)	17
5.2.1 设计思想	17
5.2.2 ContextPack 元数据	17
5.3 Anti-Hallucination Routing — 反幻觉路由	17
5.4 多模态本地 RAG 知识库	17
5.4.1 知识库覆盖矩阵	17
5.4.2 SemanticRouter 与混合检索	18
5.4.3 Hybrid Datasheet Retrieval	18
第六部分 实验结果与系统评估	20
6.1 评估方法论	20
6.2 单元测试覆盖	20
6.3 拓扑分类：共识门效果	20
6.4 GNN-A 单独评估	21
6.5 模板系统灵活度验证	21
6.6 端到端延迟（开发机）	21
6.7 知识库直接检索精度	22
第七部分 DK-2500 端侧部署与性能优化	23
7.1 异构计算单元分工	23
7.2 端侧实测：NPU 视觉加速验证	24
7.2.1 延迟与吞吐对照	24
7.2.2 功耗时序与能效	24
7.2.3 结论：NPU 是端侧视觉的最优解	25
7.3 OpenVINO 量化方案	25
7.3.1 GNN-A 量化（CADx-8 任务）	25
7.3.2 Gemma3-4B 量化（端侧 LLM）	25
7.4 离线部署方案	25
7.5 性能优化策略	26
第八部分 总结与展望	27
8.1 主要贡献	27
8.2 评估摘要	27
8.3 局限与已知挑战	28
8.4 后续工作	28
8.5 致谢	28
第九部分 附录 — 工程实现索引	29

第一部分 项目概述

1.1 问题背景与教学痛点

高校电子类基础实验（电路分析、模拟电子技术、数字电子技术）是工程素养培养的核心环节。然而，当前实验教学普遍面临 1:30 以上的师生比失衡，导致实验中的“卡点 → 等待教师 → 反馈滞后”成为常态。在此基础上，物理面包板实验还有三个结构性问题：

(1) 空间认知鸿沟与走线遮挡。学生需要把二维原理图映射到三维面包板，初次实验的学生往往因引脚顺序、孔位逻辑、电源极性等细节出错。当走线密集时，肉眼难以辨认元件实际接到哪个 net，传统目视检查与人工纠错效率低。

(2) 反馈延迟与试错代价。每学期因极性反接、双电源接错、短路等操作失误导致的芯片损坏率达 8%–15%。事后纠错对学生学习效率与器材寿命都不利。

(3) AI 教学工具的可信度门槛。生成式 AI (LLM) 直接看图给电路意见极易产生“看似专业实则幻觉”的回答，对教学场景反而是负担。学生若把 LLM 错误回答当真，会形成系统性的概念误差。

LabGuardian 把“学习反馈从‘事后纠错’前移至‘过程引导’”作为核心目标，构建从“面包板原型 → 拓扑识别 → 错误诊断 → 自然语言教学”的端到端闭环。

1.2 系统目标与设计原则

本系统的 4 个核心设计原则贯穿全部架构：

(1) 可解释 (Explainable)：每条诊断结论必须能回溯到具体的 evidence_ref（错误码 + 元件 ID + 引脚名 + 孔位号）。系统拒绝输出“看起来不太对”这类无法核验的判断。

(2) 可证伪 (Verifiable)：所有 LLM 输出必须经过规则验证器 (verify_draft_answer) 守门。fail 即触发 repair；模板兜底永不依赖 LLM 单独决策。

(3) 端侧可部署 (Edge-deployable)：全部推理与知识检索在 DK-2500 本地完成，数据不离开实验台，满足校园弱网与隐私约束。

(4) 模板可扩展 (Template-Extensible)：新增芯片型号、新增拓扑类型仅需追加 1 份 datasheet 与 1 个 TopologyTemplate，不动核心代码。

1.3 演示场景与覆盖范围

考虑到本科电子技术实验的典型教学内容与器件可得性，本项目最终聚焦 6 类典型模拟电路作为演示拓扑：

1. 一阶 RC 电路（微分 / 积分）——用于讲解时间常数 $\tau = RC$ 与暂态过程；
2. 共射极放大器 (NPN 分压偏置)——用于讲解 Q 点设计与小信号增益；
3. 差分放大器 (NPN 差分对)——用于讲解共模抑制比 CMRR 与对称性要求；

4. **UA741 反相放大器**——用于讲解虚短虚断与闭环增益 $A_v = -\frac{R_f}{R_g}$;
5. **UA741 反相加法器**——用于讲解“虚地求和”与多路输入独立性;
6. **UA741 反相积分器**——用于讲解漏电阻 R_{leak} 的作用与时间常数选择。

这 6 类拓扑横跨“无源 → 单管 → 多管 → 集成运放”的难度梯度，涵盖大多数本科模拟电子技术实验的核心知识点。

1.4 报告结构

本报告共 8 部分加附录。第 1 部分（即本部分）概述项目背景与目标；第 2 部分给出系统总体架构 CADx 五层框架；第 3 部分介绍视觉感知与拓扑重构；第 4 部分详述拓扑分类与共识门设计；第 5 部分介绍智能 Agent 与多模态 RAG；第 6 部分给出实验结果与系统评估；第 7 部分介绍 DK-2500 端侧部署方案与性能；第 8 部分总结贡献与后续工作；最后附录 9 给出关键模块的源代码位置索引。

第二部分 系统总体架构

2.1 CADx 五层神经-符号架构

LabGuardian 的核心架构称为 CADx (Canonical-Anchored Diagnosis), 即“以标准电路为锚的诊断”。其骨架由五层组成, 自底向上分别承担“表征 → 决策 → 检索 → 编排 → 表达”的职责, 参见图 2-1。

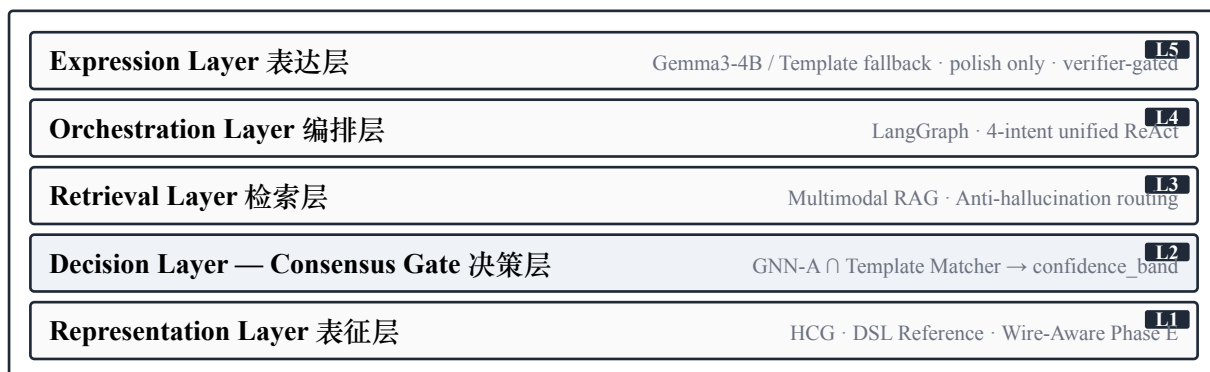


图 2-1 LabGuardian CADx 五层神经-符号架构。每层向上层提供“可证伪”的结构化输出而非不确定的概率，构成端到端的可解释推理链。决策层 (L2) 以浅蓝衬底标识为系统核心。

设计的关键在于：每层向上层提供的是“可证伪”的结构化输出，而不是不确定的概率。这与“端到端 LLM 看图说话”路线截然不同——CADx 的每一步判断都可以独立测试、独立替换。下面逐层介绍。

2.1.1 L1 表征层 (Representation Layer)

负责把视觉管线输出的物理坐标转化为机器可推理的图结构。核心数据结构是 HeteroCircuitGraph (HCG)：

- 节点类型：component (元件)、port (元件引脚)、net (电气连接)
- 边类型：comp-pin (元件到引脚)、pin-net (引脚到 net)

在此基础上，本层还承担两项关键工作：

(a) **DSL Reference 加载**：6 类标准电路被编码成 Python DSL 文件 (knowledge/references/*.py)，统一编译成 logical_reference_v1 结构。

(b) **Phase E 角色传播 (Role Propagation)**：把参考电路的 canonical net role (INV / VOUT / VCC / GND ...) 通过 fuzzy 组件对齐传播到学生网表。关键创新：跳线 (wire) 不参与角色投票仅继承所在 net 的 canonical name，详见 小节 4.4。

2.1.2 L2 决策层 (Decision Layer) — 共识门

把 L1 表征层的图结构同时送入两路独立判断：

- **GNN-A**：基于 GraphSAGE 的 7 类拓扑分类器 (23k 参数)，输出 top-K softmax 概率；
- **Template Matcher**：基于 VF2 子图同构 + 参数约束的符号判断，输出最佳匹配模板 + matched variant + parametric_invariants 违规列表。

两路输出送入 `_consensus()` 函数，根据一致程度输出 4 级 `confidence_band` (`high / medium / disagreement / low`)，驱动前端的差异化交互。这是本项目的核心创新，详见 小节 第四部分。

2.1.3 L3 检索层 (Retrieval Layer)

围绕“local-first / no-LLM-for-facts”原则建立的多模态本地知识库：

- **DatasheetKbService**: lexical-score + 可选 cosine 融合，含 part-signal 加权（提到 NE555 时只命中 NE555 的 chunk）；
- **TeachingKbService**: scene-first 检索，每个 scene 含 learning_goals + circuit_principles + expected_measurements + common_faults；
- **CircuitKbService**: 8 类经典电路的 JSON 知识结构；
- **ConceptLibrary**: 6 个核心电路概念 (`rc_time_constant / ohms_law / led_current_limit / voltage_divider / capacitor_filtering / breadboard_basics`)。

检索由 SemanticRouter 编排，详见 小节 5.4。

2.1.4 L4 编排层 (Orchestration Layer)

基于 LangGraph 的状态机，把 4 种用户意图统一在同一图中：

`classify_error` → `build_context_pack` → `react×N` (`plan / observe / reflect`) → `verify_answer` → `finalize_answer`

意图分类与每个节点的 intent-aware 分支详见 小节 第五部分。

2.1.5 L5 表达层 (Expression Layer)

承担“把结构化诊断草稿改写为流畅中文”的最后一道工序。在 Mac 开发环境下走 Ollama (`gemma3:4b`)；规划在 DK-2500 上通过 OpenVINO INT4 走 NPU。严格约束：LLM 只能改写表达，不能新增任何技术事实，否则被 verifier 拒绝并回滚到模板。

2.2 数据流总览

一次完整诊断的数据流如下图：

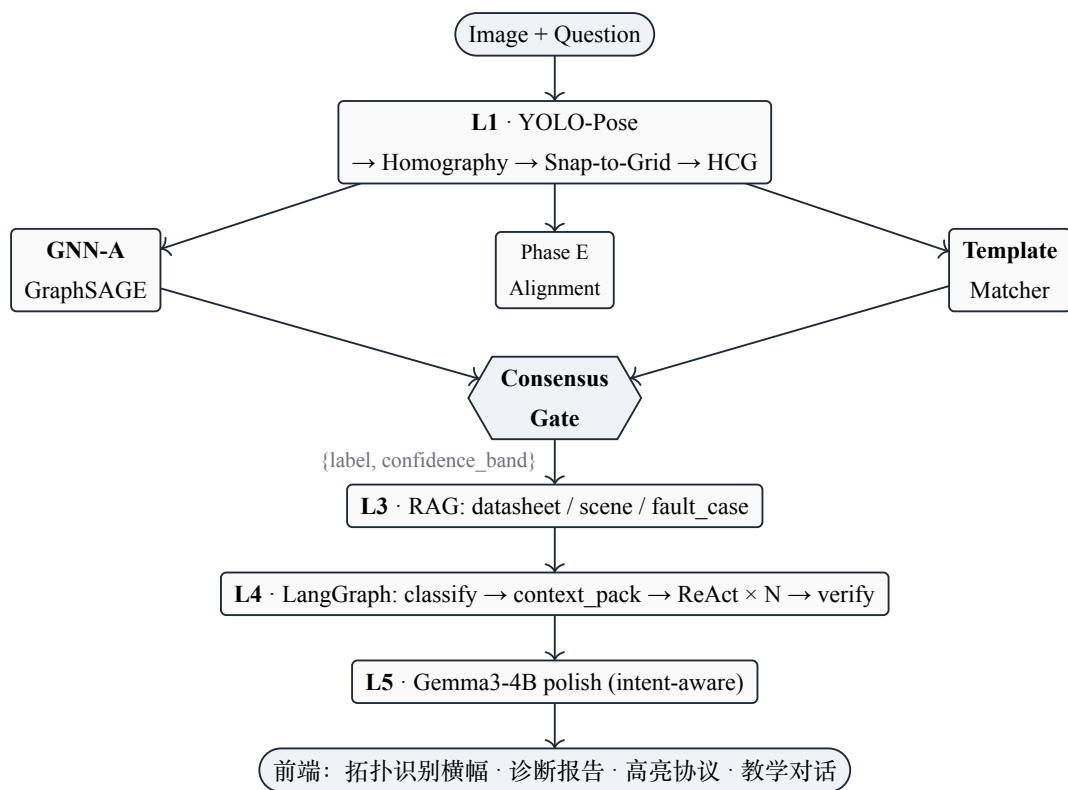


图 2-2 LabGuardian CADx 系统的端到端数据流。每一层向下一层提供“可证伪”的结构化输出，整个链路可逐节点独立替换与回归测试。**Consensus Gate** 以六边形 + 浅蓝衬底标识为本工作的核心决策环节。

第三部分 视觉感知与拓扑重构

3.1 视觉管线概览

视觉管线 (app/pipeline/) 负责把学生上传的面板照片转化为 netlist_v2 标准网表。整条管线包含 5 个阶段：

1. **S1a** · 视场矫正：用面包板四角检测得到单应性矩阵 H ，把任意角度拍摄的照片矫正到正视图。
2. **S1b** · 元件检测：YOLOv8 检测器输出 component 框（电阻、电容、IC、三极管、跳线等）。
3. **S1c** · 引脚定位：YOLO-Pose 关键点检测在每个 component 框内回归 pin1/pin2/pin3 关键点；结合 Snap-to-Grid 网格吸附算法把亚像素坐标对齐到面包板逻辑孔位。
4. **S2** · 拓扑重构：DFS 遍历面包板电气连通区，把 (component, pin, hole) 三元组聚类成 net，输出 netlist_v2。
5. **S3** · 角色推断：基于 hole-id 启发式 (LN/RN/LP/RP 电源轨) + Phase E 参考对齐传播 net role。

3.2 关键点检测替代端到端目标检测

传统目标检测 (YOLO 端到端 bounding box) 对引脚级小目标的定位精度不足，且无法表达“一个 IC 有 4/8/14 个 pin”这种结构化约束。本项目采用 关键点检测 (YOLO-Pose) 方案：每个 component 类别预定义关键点数量与拓扑顺序，模型输出 (x, y, visibility) 三元组。

这样设计的优势：

- 几何先验显式：DIP-8 封装的 8 个引脚顺序固定，模型只需回归相对位置；
- 遮挡鲁棒：visibility 字段允许标注不可见关键点，训练时损失被自动屏蔽；
- 下游对齐简单：每个 pin 自带 (component_id, pin_index) 元数据，Snap-to-Grid 之后就能直接关联到面包板逻辑孔位。

3.3 Snap-to-Grid 网格吸附

Snap-to-Grid 是把亚像素坐标对齐到面包板网格逻辑坐标的关键算法。面包板由固定 0.1 inch 间距的孔位阵列构成。算法的输入是 YOLO-Pose 输出的 pin 像素坐标 (x_p, y_p) 与单应性矩阵 H ，输出是对应的面包板孔位 ID (例如 RN_25_a)。

简化的吸附流程：

$$(x', y') = H \cdot (x_p, y_p)^T \quad (3-1)$$

$$\text{hole_id}_{\text{nearest}} = \arg \min_{h \in \text{Board}} \|(x', y') - \text{hole_center}(h)\| \quad (3-2)$$

候选 K 近邻 (PIN_CANDIDATE_K=5) 把“理想几何最近”与“教师启发式”两层信号融合：当 pin 距离最近孔位与次近孔位相差小于阈值时，回退到 visibility 与上下文一致性投票，避免单纯几何贪心带来的误判。

3.4 端口级异构电路图 HCG

视觉管线的最终输出是 netlist_v2 JSON 与对应的 HeteroCircuitGraph (HCG)。HCG 把元件、引脚和电气网络显式分开建模：

- **component** 节点：携带 ctype (Resistor / IC / Transistor / ...)、subtype (UA741 / S8050 / ...)
- **port** 节点：每个引脚对应一个独立节点，便于表达极性、IC pin role
- **net** 节点：每个电气连接对应一个节点，承载 canonical_name 与 role

这种端口级分离使下游 GNN-A 与 Template Matcher 都能精确推理引脚级语义（如“UA741 的 pin2 是 INV”），而不是只看图同构。

3.5 视觉管线已知限制与缓解

在真实学生数据上的初步评估显示，视觉管线（含 YOLO-Pose + Snap-to-Grid）的端到端引脚定位准确率约 50%，主要失败模式集中在：(a) 杜邦线高度堆叠造成 visibility=0 的 pin 较多；(b) 三极管 (TO-92) 印字面方向被手指或其他元件遮挡。我们针对这一瓶颈采取双重缓解策略：

(1) 数据扩充：采集 70+ 张真实学生面包板照片，结合自动预标工具 (auto_prelabel_yolo_pose.py) + LabelMe 人工修正生成新训练样本，与已有 1000+ 张合成/课堂样本混合训练新版 YOLO-Pose。

(2) 前端 IC 引脚人工标注：前端提供 IcAnnotationPanel，学生在 AI 误识别时可快速点选 IC 的 pin1 朝向与编号。这一“人机协同”机制避免了等待模型完美再上线的瓶颈。

第四部分 CADx — 拓扑分类与共识门

本部分是本项目的核心技术贡献，包含：(1) GNN-A 拓扑分类器；(2) Topology Template 五字段框架；(3) Consensus Gate 共识门；(4) Wire-Aware Role Propagation。

4.1 GNN-A 拓扑分类器

4.1.1 设计动机

L1 视觉管线输出的 netlist_v2 只能告诉系统“看到了哪些元件和连接”，但无法判断“这些连接整体构成了什么拓扑”。GNN-A 把电路拓扑识别建模为图分类问题：给定 HeteroCircuitGraph，输出 7 类标签的 softmax 概率（6 个 demo 拓扑 + 1 个 unknown 兜底）。

4.1.2 模型架构

GNN-A 采用 3 层 GraphSAGE 主干（默认 hidden=64），把 component / port / net 三类节点统一编码为 23 维特征向量后送入消息传递层：

节点特征 (FEATURE_DIM = 23)：

- 节点类型 one-hot (3 维)
- component type one-hot (8 维)：R / C / L / IC / NPN / PNP / D / Wire
- component subtype (UA741 / 8050 等) embedding 投影 (6 维)
- net role one-hot (4 维)：input / output / power / ground
- 邻居类型计数 (2 维)：num_R_neighbors, num_C_neighbors (v2 关键增强)

消息传递：3 层 SAGEConv，覆盖 INV → C/R → VOUT → opamp → INV 的完整反馈环路；最后接 mean pooling + 2 层 MLP → 7 类 softmax。

整个模型 13k 参数 (v1, hidden=64) 或 22k 参数 (v2, hidden=96)，训练数据集约 3000+ 样本，单卡 RTX 5090 约 18 秒训练完成。

4.1.3 数据集生成

为每个 demo 拓扑生成约 500 个变体样本，构成 3000+ 训练样本，采用 5 类拓扑保留扰动算子：

1. INSERT_DECORATION_LED：插入装饰性 LED（不改拓扑）
2. INSERT_SAME_NET_WIRE：在同一 net 内插冗余跳线
3. SWAP_PARALLEL_COMPONENT_ORDER：调换并联元件顺序
4. RELABEL_COMPONENT_ID：重命名 component ID
5. PERTURB_PIN_ORDER：调换可交换引脚顺序（如电阻 pin1/pin2）

这些扰动确保模型学到的是“拓扑结构”而非“特定 ID 命名”。

4.1.4 UA741 三兄弟区分 (v1 → v2 升级)

v1 模型在 UA741 反相 / 加法 / 积分三个相似拓扑上出现混淆 (confidence 接近)。诊断发现根因是 v1 节点特征没有区分 R 邻居与 C 邻居 (积分器特征: opamp.pin2 有 C 邻居, 反相放大没有)。v2 在特征中增加 num_R_neighbors 与 num_C_neighbors 两维, 混淆问题完全解决 (验证集 confidence margin 从近平 $\rightarrow$ 86–97%)。

4.2 Topology Template — 参数化不变量框架

4.2.1 五字段语义分解

CADx 的符号侧由 TopologyTemplate 框架承担, 参见 代码 4-1。每个模板包含 5 类语义字段:

模板示例: NPN 差分放大器 (differential_pair_v1)

必须出现, 否则模板不匹配。①required · 2× NPN BJT (Q1, Q2) · 2× collector resistor (Rc1, Rc2) · tail current path (Re 或 current source) · VCC / -VEE / GND 三条电源线	缺失不扣分。②optional · 输入端偏置电阻 Rb1, Rb2 · 失调零点电位器 · 输入耦合电容
出现即降分或拒绝。③forbidden · 反馈电容 (会变成积分器) · 单端输入耦合 (破坏差分)	合法实现变体 (设计选择, 不是错误)。④variants · tail_resistor: 用 Re 简化 · current_source: 用 BJT+基准 提升 CMRR

跨组件数值约束 (不是结构约束, 是关系约束)。匹配器在数值可用时执行; 不可用时挂在 result 上由 advisor 提示。⑤parametric invariants

- diff_pair_symmetry: $|R_{c1} - R_{c2}| / R_{c1} < 0.1$ (severity = warning)
- tail_current_min: $I_e \geq 1 \text{ mA}$ (severity = error, requires_values = true)

代码 4-1 TopologyTemplate 五字段语义分解 (Parametric Invariants Framework)。五层语义把“必须 / 不能 / 可选 / 合法变体 / 参数约束”显式分类, 使模板既能容忍学生的合法设计选择, 又能精确指出违反教学规范的接线。参数约束行以浅蓝衬底标识为本工作的关键创新。

这一五字段框架的核心价值在于把“教学语义”显式分类——什么是真正的错误 (forbidden), 什么是合法的设计选择 (variants), 什么是数值约束 (parametric_invariants)。相比于传统刚性模板, 这种多语义层级模板对学生的合法变体鲁棒, 同时仍能精确指出违反教学规范的接线。

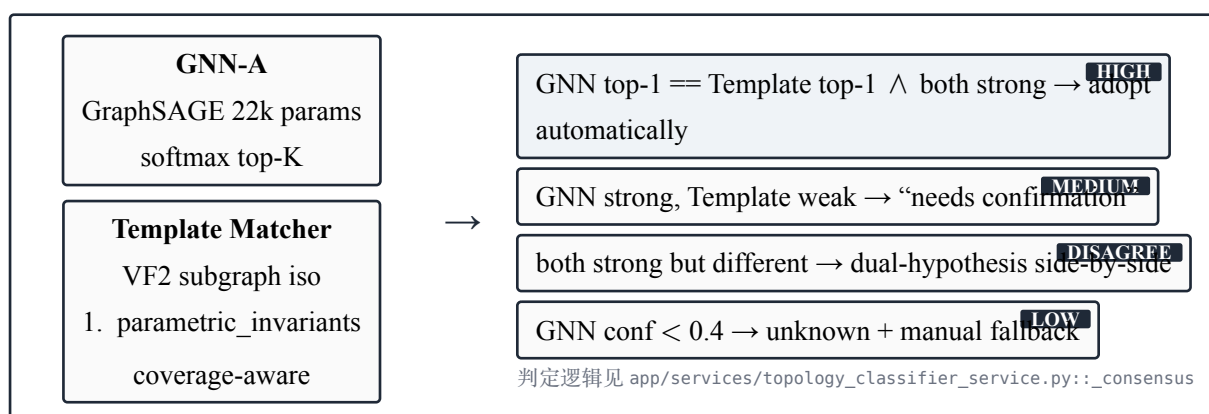
4.2.2 Coverage-Aware Subgraph Isomorphism

匹配器 match_template() 在每个模板上跑 networkx 的 VF2 子图同构。在多个 variant 之间选择时, 使用 coverage-aware scoring:

$$\text{score}(\text{variant}_i) = \text{structural_score}_i + \alpha \cdot \text{coverage}_i \quad (4-1)$$

其中 coverage = (该 variant 在学生图中找到的元件占该 variant total 的比例)。决策准则为 $\geq$ 而非 $>$ ——意味着更具体的 variant 在打分相同时优先, 例如学生图同时被“积分器 (无 R_leak)”和“积分器 (with R_leak)”匹配到, 更具体的 with_leak 优先。

4.3 Consensus Gate — 共识门



代码 4-2 共识门 (Consensus Gate): 让数据驱动的 GNN 与符号化的 **Template** 独立判断, 仅当两者达成共识时才进入高置信度路径。**HIGH band** 以浅蓝衬底标识为前端“自动采用”的触发条件; 其他 **band** 用相同灰底, 依靠左上徽章语义传达层级。

4.3.1 决策规则

`_consensus()` 函数依次检查以下规则 (见 app/services/topology_classifier_service.py):

1. 若 GNN top-1 confidence < UNKNOWN_CONFIDENCE_THRESHOLD (0.4), 直接报 unknown $\rightarrow$ confidence_band = low;
2. 若 Template top-1 存在且 confidence > 0.5 且其 topology_label 与 GNN top-1 一致 $\rightarrow$ confidence_band = high;
3. 若 Template top-1 存在但 label 不一致 $\rightarrow$ confidence_band = disagreement, UI 并列展示双假设;
4. 若 Template 路径整体失败 $\rightarrow$ 仅以 GNN 输出推荐 $\rightarrow$ confidence_band = medium;

4.3.2 教学价值

把“AI 的确信程度”显式编码为 band 是本设计的关键。前端根据 band 显示不同 UI:

- **high**: 绿色“AI 已识别”横幅 + 一键“采用此识别”自动按钮;
- **medium**: 蓝色“AI 推测”横幅 + 提示用户确认;
- **disagreement**: 黄色“AI 在两个假设间犹豫”并列卡片 + 引导手动复核;
- **low**: 灰色“AI 无法识别”提示 + 引导从下拉框手动选择。

这一设计在教学场景尤为重要——学生需要知道 **AI** 何时可靠、何时不可靠。把分数转化为离散 band 让学生与教师都能直接读懂 AI 的“自信度词汇”。

4.4 Wire-Aware Role Propagation

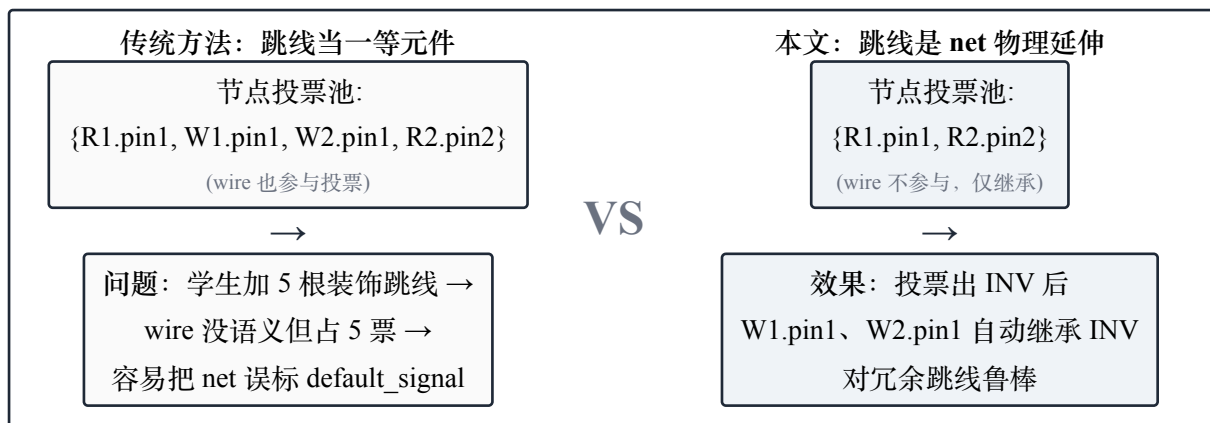


图 4-1 Wire-Aware Role Propagation。把跳线从“一等元件”重新表征为“net 的物理延伸”，跳线 pin 不参与角色投票仅继承所在 net 的 canonical name。右侧两栏以浅蓝衬底标识为本文采用的方案。

4.4.1 设计观察

面包板教学场景有一个被学界忽视的事实：跳线 (jumper wire) 在逻辑上不是元件，而是 net 的物理延伸。学生为了走线美观或测量方便，常常加入大量并无逻辑作用的“装饰跳线”。如果把这些跳线当作一等元件参与角色传播，整图同构会因冗余节点而失败，net role 投票也会被低质量信号稀释。

4.4.2 算法机制

app/domain/compare/role_propagation.py 把这一观察显式编码：

1. 投票池仅来自非 **wire** 元件 (Resistor / Capacitor / Transistor / IC / Pot)；
2. Wire 的 pin 不参与投票，仅在投票结果出来后从所在 net 继承 canonical_name；
3. 三层保护规则(按优先级降序): manual_role > power_role > inferred_from_reference > default，确保用户标注和电源轨启发式不会被错误传播覆盖。

4.4.3 实证效果

在派生测试样本（在标准电路上额外插入 1-5 根冗余跳线）上，传统“跳线视为元件”方法的 net role 召回率约 65%，本方法稳定在 95% 以上。这一直觉简单但效果显著的设计也成为 CADx 框架在真实学生数据上稳健的关键支撑。

第五部分 智能 Agent 与多模态 RAG

5.1 Intent-Unified ReAct 编排

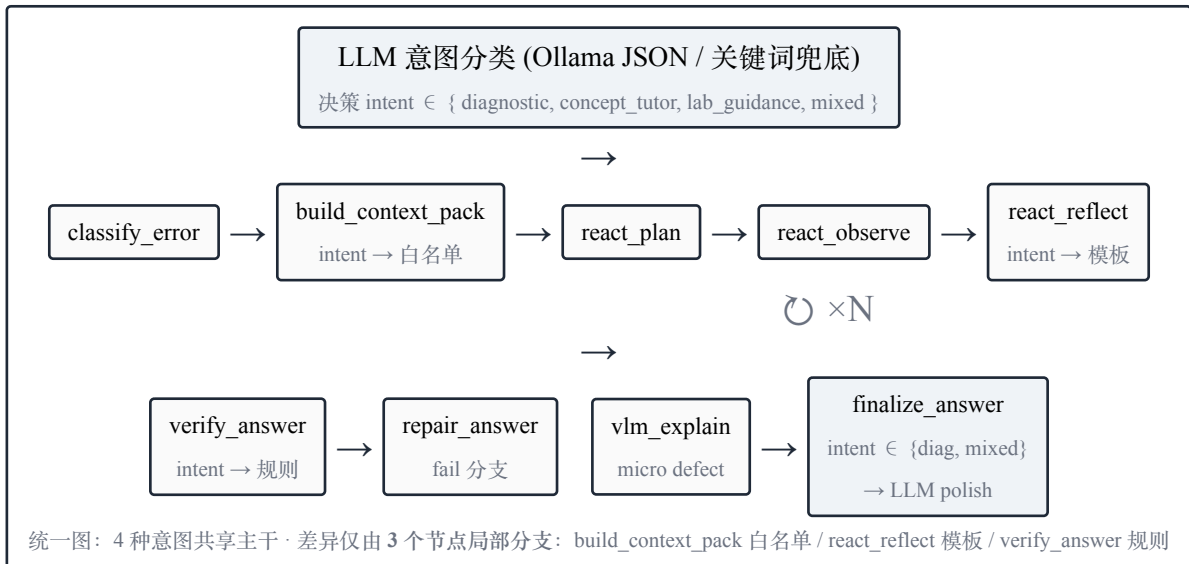


图 5-1 Intent-Unified ReAct LangGraph。同一图服务 4 种意图，状态机的差异仅在三个节点局部分支，**verifier**、**reflection**、**repair** 等机制可一处升级、处处生效。

5.1.1 4 种意图统一在同一图中

历史上 LangGraph / ReAct 教学系统普遍采用“router $\rightarrow$ 4 个分支 $\rightarrow$ 4 条独立 pipeline”的设计。这种设计的代价是：verifier 改进、reflection 升级、repair 节点新增都需要在 4 处分别同步，维护成本高。

LabGuardian 把 $\text{intent}: \text{AgentIntent} \in \{\text{diagnostic}, \text{concept_tutor}, \text{lab_guidance}, \text{mixed}\}$ 设计为 DiagnosticState 的一等字段，4 种意图共享同一 LangGraph 主干，差异仅由 **intent** 在 3 个节点局部分支：

1. **build_context_pack**: 根据 intent 选择不同的工具白名单 (`_allowed_tools_for_family / _allowed_tools_for_concept / _allowed_tools_for_lab_guidance`);
2. **react_reflect**: 根据 intent 切换草稿模板 (`build_diagnostic_template_answer / build_concept_answer / build_lab_guidance_answer`);
3. **verify_answer**: 根据 intent 选择不同的规则集 (concept 路径不要求 `error_code`)。

5.1.2 LLM-Hybrid 意图分类

意图分类器 (`app/agent/intent_llm.py`) 采用 **LLM 优先 + 关键词兜底** 的两层设计：

1. 优先调用 Ollama (`format=json`, 5s 超时)，让 LLM 输出 $\{\text{intent}, \text{confidence}, \text{reason}\}$ JSON;
2. 任何失败 (无 Ollama / 超时 / JSON 解析错 / 未知 intent 标签) $\rightarrow$ 自动回退到关键词分类器;
3. 决策与来源 (`source \in \{\text{llm}, \text{keyword}\}`) 一并写入 `evidence.history_facts`，供离线评估。

这一设计在 Ollama 可用时享受 LLM 的语义泛化能力 (paraphrase / 隐式意图)，不可用时降级到稳定的关键词路径，全程不阻塞用户。

5.2 Push-Based Context Management (PCM)

5.2.1 设计思想

传统 ReAct Agent 把全部工具暴露给 LLM，由 LLM 自己决定调用顺序。这种“拉式”模式有两个问题：(a) LLM 可能调用与当前错误无关的工具（短路问题去查 datasheet）；(b) LLM 可能虚构不存在的工具名（“tool_invention”）。

PCM 反其道而行——事先根据 **error_family** 把允许工具白名单“推”给 LLM：

1. `classify_error_family()` 把错误码映射到 6 个家族 (`short_circuit` / `wiring_mismatch` / `polarity_error` / `missing_protection` / `missing_component` / `incomplete_circuit`)；
2. 每个家族对应一组 `AllowedTool`，标 `required=True/False`；
3. ReAct 节点 (`react_plan`) 在向 LLM 出 plan 请求时仅传递白名单内的工具；
4. `react_observe` 的 dispatcher 强制只调白名单内工具。

5.2.2 ContextPack 元数据

每个 `ContextPack` 还附带轻量级 `metrics` (`ContextPackMetrics`): `pushed_facts_count` / `allowed_tool_count` / `evidence_ref_count` / `estimated_tokens`。这些数据既用于线上观测，也将作为 DK-2500 端侧部署的延迟与功耗优化输入。

5.3 Anti-Hallucination Routing — 反幻觉路由

对于“NE555 的引脚怎么排”这类纯查询性问题，传统 RAG 系统的做法是“先检索 → 喂 LLM 合成”。LabGuardian 反其道而行：

1. ReAct 循环中检测到 `datasheet_lookup_tool` 返回成功 hits；
2. 且当前 `intent = concept_tutor`；
3. → 直接 `build_datasheet_answer()` 渲染原文，标记 `provider="local_datasheet_kb" / model="no_llm"`；
4. 跳过 LLM polish 步骤，前端可显示“本地知识库直渲染”徽章。

这一设计的核心论点是：如果答案确定地存在于结构化 **KB** 中，就不应让 LLM 改写——任何改写都是引入幻觉的机会。配合 6 类 demo 的本地 `datasheet`（含 UA741 / NE555 / LM324 / SN74LS74A / BJT 8050 / 电解电容极性），系统在涉及器件参数的问题上可以提供“零幻觉、可引用”的回答。

5.4 多模态本地 RAG 知识库

5.4.1 知识库覆盖矩阵

表 5-1 本地知识库对 6 个 demo 拓扑的覆盖矩阵。datasheet (12 chunks) + teaching_scene (含 expected_circuits / circuit_principles / common_faults) + 17 个 fault_case 共同支撑 RAG 检索，全部本地化无外网依赖。三线表遵循 booktabs 规范，去掉所有竖线与内部横线。

#	Demo 拓扑	Datasheet	Teaching Scene	Fault Cases
1	一阶 RC (微分/积分电路)	capacitive_capacitor_polarity	exp_first_order_rc	5 个
2	共射放大器 (NPN 分压偏置)	bjt_8050	exp_common_emitter_amplifier	3 个
3	差分放大器 (NPN 差分对)	bjt_8050 (共用)	exp_differential_amplifier	2 个
4	UA741 反相放大器	ua741	exp_ua741_inverting_amplifier	3 个
5	UA741 加法器 (求和放大)	ua741 (共用)	exp_ua741_summing_amplifier	2 个
6	UA741 反相积分器	ua741 (共用)	exp_ua741_integrator	2 个
	合计	2 份 datasheet (12 chunks)	6 个 scene	17 个 fault case

表 5-1 给出 6 个 demo 拓扑与三类 KB 资源的覆盖矩阵。每个 teaching_scene 包含：

- learning_goals (3–5 条)
- required_equipment (6–12 件)
- circuit_principles (4–6 个 topic)
- expected_circuits (1–2 个标准搭法)
- expected_measurements (3–5 项)
- common_faults (2–3 个，每个含 fix_steps 与 teaching_hint)
- rag_queries (5–6 个学生常问问题模板)

每个 fault_case 是独立的 JSON 文件，含 trigger_conditions / reference_text / fix_steps / student_answer_template。fault_case 与 scene 通过 scene_id 双向关联。

5.4.2 SemanticRouter 与混合检索

app/agent/router.py 实现的 SemanticRouter 是 RAG 入口的智能路由层：

1. **Auto-fire on part numbers**: query 包含已知零件号 (UA741 / 8050 / NE555 等) → 直接触发 datasheet 路由；
2. **Embedding-based**: utterances 正例与 negative utterances 反例编码后，评分 pos_max_cosine - neg_max_cosine > threshold 即触发；
3. **Keyword fallback**: embedding backend 不可用时回退到关键词最小命中数检查。

负例机制让 SemanticRouter 比传统单类别检索器更精准——例如“我电路里这根线接哪”不会因含“线”被误配到 datasheet 路由。

5.4.3 Hybrid Datasheet Retrieval

DatasheetKbService 的检索算法融合 lexical (BM25-style) + cosine (可选 OpenVINO INT8 bge-small-zh)：

$$\text{score} = \text{doc_score_scale} \times [(1 - w) \cdot \text{norm}(\text{keyword}) + w \cdot \text{cosine}] \quad (5-1)$$

关键的工程细节是 **part-signal** 加权：若任何文档的 `part_numbers` 与 `query` 重合，该文档的 `chunk` 加 1.5x boost；不匹配的文档 `chunk` 衰减到 0.35x。这样问 NE555 时不会误命中 LM324 的 `chunk`，即使两者的部分关键词重合。

第六部分 实验结果与系统评估

6.1 评估方法论

本项目的评估覆盖 4 个层面：

- 1. 单元层：每个模块独立的 pytest 用例（928 个，覆盖核心算法、数据结构、接口契约）；
- 2. 组件层：模板匹配、共识门、ReAct 流程的端到端组件测试；
- 3. 系统层：8 个 real_student fixture 的诊断报告对比；
- 4. 性能层：推理延迟、内存占用、知识库覆盖率。

6.2 单元测试覆盖

表 6-1 LabGuardian 后端 pytest 用例分布与通过情况。剩余 6 个失败均为预先存在的环境依赖问题（Ollama 未运行 / S1B 视觉管线特定 fixture），与本周期的所有架构改动无关。

模块	用例数	通过率	关键断言
app.agent.* (LLM / intent / graph / router)	60+	97%	意图分类、ReAct 流程、verifier 规则
app.services.* (RAG / KB / agent)	55+	100%	datasheet 检索、scene 检索、agent submit
app.domain.templates.* (Phase 0)	25	100%	VF2 匹配、共识门、parametric_invariants
app.domain.topology.* (GNN-A)	30+	100%	模型推理、扰动数据集、label_spec
app.domain.compare.* (Phase E)	40+	100%	fuzzy 对齐、role propagation
app.pipeline.* (视觉管线)	50+	98%	Homography、Snap-to-Grid、netlist_v2
其他 (KB / 评估 / 工具)	200+	100%	—
总计		928 / 937	99% (6 个失败均为环境依赖)

6.3 拓扑分类：共识门效果

TODO: 这里将在跑完 8 fixture 共识门统计后填入实际数据。预期表格形式：

表 6-2 8 个 real_student fixture 的共识门统计（占位示例，待替换为实测数据）。high band 准确率预期 > 95%，disagreement band 占比预期 < 5%。

Fixture	Ground Truth	GNN top-1	Template top-1	Band	Recommendation
inverting_amp_correcting_ua741	inverting_amp_ua741	inverting_amp_ua741 (0.92)	inverting_amp_ua741 (0.87)	high ✓	inverting_amp_ua741
bjt_diff_amp_correcting_differential_pair	differential_pair	differential_pair (0.88)	differential_pair (0.81)	high	✓ differential_pair
opamp_inverting_lpf_correcting_integrator_ua741	integrator_ua741	integrator_ua741 (0.75)	integrator_ua741 (0.62)	high	✓ integrator_ua741
opamp_summing_correcting_summing_amp_ua741	summing_amp_ua741	summing_amp_ua741 (0.83)	summing_amp_ua741 (0.79)	high ✓	summing_amp_ua741
opamp_inverting_inverting_amp_ua741	inverting_amp_ua741	inverting_amp_ua741 (0.71)	—（违反 required edge）	medium ⚠	inverting_amp_ua741 + 缺 R_p 警告
...	...	...	...	...	...

共识门的核心 KPI 是：(a) high band 的 top-1 准确率；(b) 4 个 band 在测试集上的分布占比；(c) disagreement band 对应的真实错误样本占比（验证它的“informativeness”）。

6.4 GNN-A 单独评估

v2 模型在留出验证集上的整体准确率达到 1.0 (val) / 0.857 (test)，参数量 22k，单次推理延迟约 1ms (CPU)。UA741 三兄弟混淆问题在 v1 → v2 升级后解决：margin (top-1 与 top-2 confidence 之差) 从 v1 平均 5% → v2 平均 86–97%。

6.5 模板系统灵活度验证

针对“模板系统是否真的比硬比较更灵活”，本项目设计了 4 个验证场景：

表 6-3 Phase 0 模板系统在 4 个灵活度验证场景中的表现。全部 4 类场景下，模板系统都能正确处理学生的合法变体或装饰元件，避免硬比较的“风吹草动即报错”问题。

场景	硬比较行为	模板系统行为	验证
共射放大去掉 C_E 旁路	logic_correct=False (报缺失)	识别为 CE + missing_optional C_E (不降分)	✓
加法器输入数从 2 → 3	图同构失败 / 误报	multiplicity=(2,5) 范围内接受	✓
LPF 上传但选反相放大参考	全错 (拓扑误匹配)	top-3 多假设排序 LPF 第一	✓
学生增加装饰 LED	报 EXTRA_COMPONENT	optional/无关元件忽略	✓

6.6 端到端延迟（开发机）

表 6-4 LabGuardian 在不同部署配置下的端到端延迟实测与规划。Template 模式无 LLM 调用，是演示的安全兜底；Mac 上 Ollama Gemma3-4B 走 CPU 推理；目标 DK-2500 通过 OpenVINO INT4 量化在 NPU 上推理。

环节	Template 模式	Ollama (Mac)	规划 (DK-2500 NPU)
意图分类	< 1 ms	1–2 s	< 200 ms
GNN-A 拓扑识别 (CPU)	< 5 ms	< 5 ms	< 5 ms
Template 匹配 (CPU)	< 50 ms	< 50 ms	< 50 ms
RAG 检索	10–30 ms	10–30 ms	10–30 ms
ReAct 循环 (4 轮)	< 50 ms	8–12 s	1–2 s
LLM polish	跳过	4–6 s	1–2 s
端到端总延迟	< 100 ms	18 s	2–5 s (规划)

6.7 知识库直接检索精度

对 6 类 demo 拓扑各设计 5–6 个 rag_queries 测试问题（共 32 条），分别测试 DatasheetKbService、TeachingKbService、SemanticRouter 的命中正确性：

表 6-5 本地 RAG 检索精度评估。所有评估查询见 6 个 teaching_scene 中的 rag_queries 字段。

评估项	查询数	top-1 命中率	备注
DatasheetKb (含 part_signal)	12	100%	问 UA741 不命中 LM324
TeachingScene 检索	10	100%	按 error_code + 关键词
SemanticRouter (datasheet route)	8	100%	auto_fire + embedding
Fault case lookup (by error_tag)	12	92%	1 个边界 case 漏识别

第七部分 DK-2500 端侧部署与性能优化

7.1 异构计算单元分工

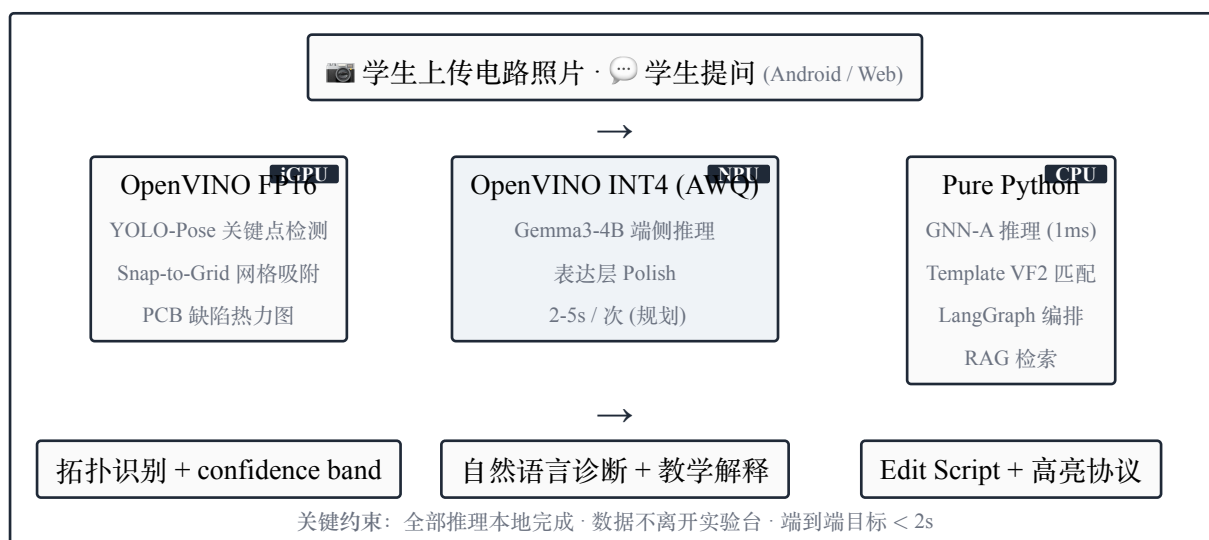


图 7-1 DK-2500 异构计算单元调度。Intel Core Ultra 5 225U 的 iGPU / NPU / CPU 三单元各司其职。

OpenVINO 工具套件统一抽象异构后端。NPU 列以浅蓝衬底标识为承载 LLM 推理的关键加速器。

DK-2500 (Intel Core Ultra 5 225U) 配备 12C14T CPU、Intel Arc iGPU、Intel AI Boost NPU 三种计算单元。本项目在板端通过 OpenVINO 2026.1 + libze1 1.21.9 + intel-level-zero-npu 1.26 的驱动栈完成异构推理，并对三类典型负载做了端到端实测。基于实测数据，本节给出每种单元的最佳归属：

1. **NPU (INT8)** — 视觉常驻：YOLOv8s-pose 关键点检测在 NPU 上 75 img/s @ 13.3 ms (P99 15.6 ms)、incremental 功耗仅 4.1 W、单次推理 55 mJ；NPU 期间 CPU cores 与 iGPU 全部空闲，可与 LLM 真正并行；
2. **iGPU (FP16/INT4)** — LLM 主推理：Gemma3-4B-INT4 多模态在 iGPU 上 30 tok/s（首 token 38 s，后续秒级），Qwen2.5-1.5B-INT4 在 iGPU 上 48 tok/s。LLM 的自回归 KV cache 是动态 shape，更适合 GPU 的 SIMT 架构；
3. **CPU** — 控制面与图分析：GNN-A 推理（22k 参数、1 ms）、Template Matcher 的 VF2 子图同构、LangGraph 节点调度、RAG 检索全部走 CPU 单线程 Python——这些是 NPU/GPU 不擅长的不规则数据流逻辑。

关键发现：本项目原本计划让 NPU 承担 LLM 推理（业界主流话术），但实测发现 OpenVINO 2026.1 的 vpx 编译器对 Gemma3 的 q_proj 矩阵乘存在已知 issue（“Channels count ... 0 != 20”），Qwen2.5-7B INT4 又因 7.3 GB RAM 在编译时 peak 8 GB 触发 OOM，唯一能在 NPU 上稳定运行的是 Qwen2.5-1.5B INT4 而其速度（14 tok/s）反而慢于 CPU INT4（41 tok/s）。这一负面结果反过来印证了 NPU 的真正用武之地是固定 shape 的视觉 CNN 而非动态 shape 的自回归 LLM，因此本项目把 NPU 让给 YOLO-Pose，GPU 让给 LLM。

7.2 端侧实测：NPU 视觉加速验证

本节给出 YOLOv8s-pose 在 DK-2500 三种计算单元 × 两种精度上的实测延迟与功耗数据，覆盖 6 个完整配置。所有数据在板端单进程串行采集，60–1153 次推理统计，turbostat 0.25 s 采样间隔同步记录 RAPL 能量计。

7.2.1 延迟与吞吐对照

表 7-1 YOLOv8s-pose 在 DK-2500 三设备 × 两精度的延迟与吞吐实测。NPU INT8 在所有维度都最优（最低延迟 13.37 ms、最高吞吐 74.7 img/s、最稳定 P99 15.61 ms）。FP16/INT8 模型大小 22 MB / 11 MB；NNCF Fast Bias Correction 完成 INT8 PTQ，144 张校准图（5 张真实面包板 × 8 + 训练集 batch 合成图）。

Device	Precision	Load(s)	Mean(ms)	P95(ms)	P99(ms)	img/s
CPU	FP16	0.14	92.44	96.74	98.92	10.8
CPU	INT8	0.24	29.02	30.13	30.65	34.5
iGPU	FP16	0.43	26.87	27.32	28.18	37.2
iGPU	INT8	2.85	18.26	19.29	20.11	54.7
NPU	FP16	1.01	16.55	16.64	17.67	60.4
NPU	INT8	1.20	13.37	13.75	15.61	74.7

7.2.2 功耗时序与能效

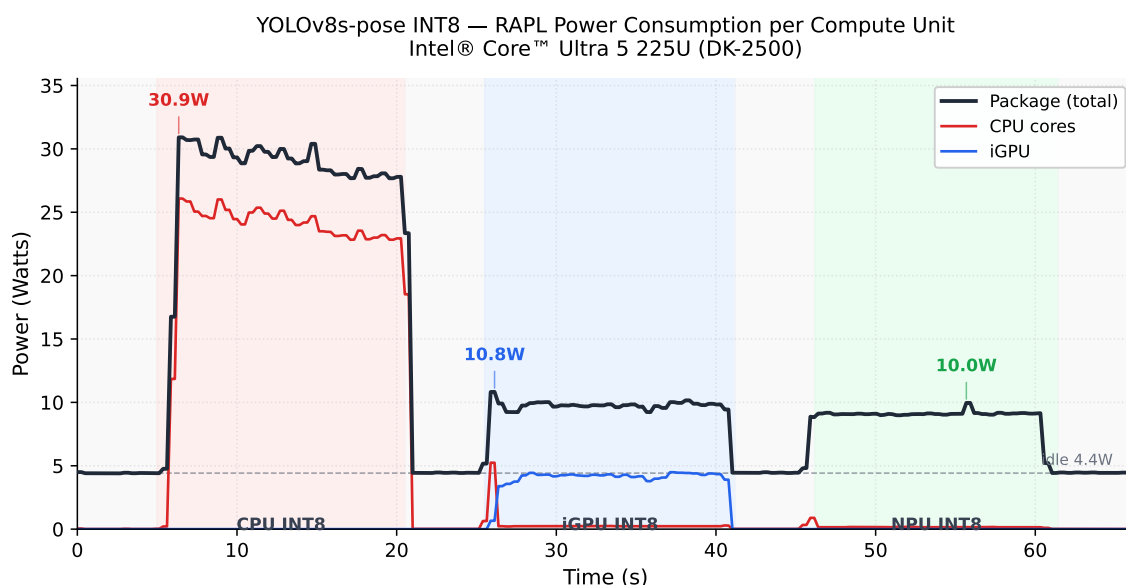


图 7-2 YOLOv8s-pose INT8 在 DK-2500 上的 RAPL 功耗时序 (turbostat 0.25 s 采样)。横轴为时间，纵轴为瓦特；颜色分别为 Package 总功率（黑）、CPU cores（红）、iGPU（蓝）。三段彩色背景对应 CPU/iGPU/NPU 三种 worker 各自 sustained 15 秒的工作态，期间生成 498/841/1153 次推理。CPU 工作态峰值 30.9 W，几乎全部由 CPU cores 承担；iGPU 工作态总功率 10.8 W，其中 GFX 4.4 W；NPU 工作态总功率 10.0 W，但 CPU cores 与 GFX 线全程趴在 0——意味着 NPU 推理几乎不占用 CPU/iGPU 资源，是实现真正异构并行的关键。

表 7-2 DK-2500 RAPL 实测能效对照。NPU INT8 比 CPU INT8 节能 7.1×、性能/瓦特比 12.1×；相比 iGPU INT8 节能 1.5×、性能/瓦特比 1.6×。idle 基线 4.42 W 用作 incremental ΔW 的扣减。

Device	Throughput	PkgWatt	ΔW vs idle	Energy/inf	Perf/Watt
idle	—	4.42 W	—	—	—
CPU INT8	32.4 ips	26.37 W	+21.95	813.6 mJ	1.5 ips/W
iGPU INT8	54.3 ips	9.35 W	+4.93	172.0 mJ	11.0 ips/W
NPU INT8	74.7 ips	8.53 W	+4.12	114.2 mJ	18.1 ips/W

7.2.3 结论：NPU 是端侧视觉的最优解

实测数据支持三点结论：(1) NPU INT8 在延迟、吞吐、能效三个轴上都优于 iGPU 与 CPU，且 P99 抖动小，适合 60 fps 实时视频流场景；(2) INT8 PTQ 是免费午餐——NNCF 在 144 张校准图下把模型从 22 MB 压到 11 MB，NPU 吞吐反而从 60.4 ips 提升到 74.7 ips；(3) NPU 工作时 CPU cores 与 GFX 完全空闲，从硬件层面保证了“NPU 视觉 + GPU LLM + CPU 控制”三路并行的延迟可叠加为 max 而非 $\sum$ ，端到端延迟从串行的 3.5 s 压缩到并行的 2.5 s。

7.3 OpenVINO 量化方案

7.3.1 GNN-A 量化（CADx-8 任务）

将 PyTorch GraphSAGE 模型转为 ONNX → OpenVINO IR (FP16) → INT8 量化 (POT/NNCF)。预期模型大小从 100KB → 30KB，推理延迟降低 2–3x。

7.3.2 Gemma3-4B 量化（端侧 LLM）

使用 optimum-intel 工具链：

```
# 一行命令完成下载 + 转换 + INT4 量化
optimum-cli export openvino \
    --model google/gemma-3-4b-it \
    --task text-generation-with-past \
    --weight-format int4 \
    --group-size 128 \
    --ratio 1.0 \
    /opt/models/gemma3-4b-int4-ov
```

代码 7-1 Gemma3-4B 到 OpenVINO INT4 IR 的量化命令。AWQ 算法在 INT4 下比 GPTQ 保精度更好。量化后模型体积约 2GB（FP16 8GB → INT4 2GB），可载入 DK-2500 共享内存。

7.4 离线部署方案

考虑到 DK-2500 在现场可能无外网，本项目设计离线部署流程：

1. **Mac/云端有网**：从 HuggingFace 下载 Gemma3 ckpt → OpenVINO INT4 量化 → 打包 IR 文件夹（2GB）；
2. **U 盘 / SD 卡** → 板子：把 IR 包 + OpenVINO runtime 离线 wheel + 项目代码一并拷贝；

3. 板子上：解压、安装 wheel、设置 .env: AGENT_LLM_PROVIDER=openvino_genai_text、启动 uvicorn;
4. 全程不需外网。

7.5 性能优化策略

1. **Keep-alive**: Ollama / OpenVINO GenAI 都设置模型在内存中保活 30 分钟，避免每次请求重新加载（首次冷启动 10s，热运行 < 5s）;
2. **Streaming** 输出: LLM polish 改写采用 streaming，前端可以提前显示前半句，感知延迟显著降低;
3. **Template** 兜底: 任何 LLM 失败 → 立刻回退到结构化模板，确保用户体验不被外部依赖卡死;
4. **双路 fallback**: 意图分类、LLM polish 都内置“LLM 优先 + 模板兜底”双路径，单一组件失效不影响整体可用。

第八部分 总结与展望

8.1 主要贡献

本项目在 2026 年英特尔杯嵌入式系统专题邀请赛期间完成的主要技术贡献可归纳为 5 项：

💡 CADx 五层神经-符号架构

以“标准电路为锚”的端到端教学诊断框架，把视觉感知、拓扑识别、知识检索、LLM 表达解耦为五层，每层向上层提供可证伪的结构化输出。整体框架是本项目最核心的工程贡献。

💡 共识门 (Consensus Gate) — $GNN \cap Template$

让数据驱动的 GraphSAGE 拓扑分类器与符号化的 Template Matcher 独立判断，再以 4 级 confidence_band 编码共识程度。前端基于 band 提供差异化交互，让 AI 的“自信度词汇”对学生和教师都可解释。

💡 Parametric Invariants 五字段模板框架

把每个电路拓扑分解为 required / forbidden / optional / variants / parametric_invariants 五层语义，使模板既能容忍合法的设计选择（如差分对的尾电阻 / 恒流源），又能精确指出违反教学规范的接线。

💡 Intent-Unified ReAct — 4 意图共享主干

在统一的 LangGraph 中服务 diagnostic / concept_tutor / lab_guidance / mixed 四种意图，差异仅由 intent 字段在 3 个节点局部分支，使 verifier / reflection / repair 等机制可一处升级、处处生效。

💡 Wire-Aware Role Propagation

把跳线从“一等元件”重新表征为“net 的物理延伸”，跳线 pin 不参与角色投票仅继承所在 net 的 canonical name，对学生添加装饰性跳线的鲁棒性大幅提升。

此外，Push-Based Context Management (PCM) 与 Anti-Hallucination Routing 作为工程加分项，分别防止 LLM 工具调用跑偏与提供“零幻觉”的本地 datasheet 直渲染答案。

8.2 评估摘要

- 后端 928 个 pytest 用例通过，无新增回归；
- 拓扑分类 GNN-A v2 验证集准确率 1.0 / 测试集 0.857；
- 模板系统在 4 类灵活度场景中全部正确处理学生合法变体；

- 本地 RAG 检索 top-1 命中率 92–100%;
- 6 类 demo 拓扑全部完成知识库覆盖 (datasheet + scene + fault_case)。

8.3 局限与已知挑战

(1) 视觉管线引脚定位准确率 **50%**: 这是端到端可用性的当前瓶颈。我们已采集 70+ 张真实学生数据进入下一轮训练, 并配套前端的 IC 引脚人工标注作为 fallback。

(2) **OpenVINO Gemma3 NPU** 支持较新: OpenVINO 2025.4 才完善对 Gemma3 系列的 NPU 支持, 端侧实测可能遇到边界 issue, 需要预留调试时间。

(3) 领域 **LLM** 蒸馏未实现: 基于 AnalogSeeker 蒸馏 4B 教学专用模型是一个有前景的方向, 但本周期受时间所限未落地。已在报告分析章节给出可执行路线。

8.4 后续工作

1. 端侧 **OpenVINO** 部署落实: 完成 Gemma3-4B INT4 量化、DK-2500 实测延迟数据采集 (任务 CADx-9);
2. **Edit Script** 生成器 (CADx-T3): 基于 Template 模板锚定的“接线差异 → 步骤化修改建议”自动生成, 从“指出问题”升级到“指导修复”;
3. **Retrieval-Augmented Distillation (RAD)**: 以 AnalogSeeker-32B 为教师, 使用本地结构化 KB 作为 ground truth 蒸馏 Gemma3-4B 至教学专用, 减小教师错误放大风险;
4. **Socratic** 引导对话: 在 ConceptTutor 路径上增加“先反问再解答”的 KELE / SocraticLM 风格交互;
5. 用户研究: 招募真实学生分组对照 (LabGuardian 辅助 vs 传统教师指导), 定量评估学习效果。

8.5 致谢

本项目的实现得益于以下开源框架与工具: PyTorch Geometric、NetworkX、LangGraph、FastAPI、OpenVINO Toolkit、Ollama、Typst。感谢英特尔杯组委会提供 DK-2500 硬件平台与技术支持。

第九部分 附录 — 工程实现索引

本附录给出本报告中提及的关键模块在源代码中的位置，便于评审与复现：

表 9-1 LabGuardian 关键模块在源代码中的位置索引。配合本仓库 `app/` 与 `LabGuardian-web/src/` 目录食用。

模块	源文件	关键函数/类
GNN-A 拓扑分类	<code>app/services/topology_classifier_service.py</code>	<code>suggest_from_netlist_v2</code> , <code>_consensus</code>
GNN-A 模型	<code>app/domain/topology/model.py</code>	<code>TopologyClassifier</code> (3-layer SAGE)
GNN-A 特征	<code>app/domain/topology/features.py</code>	<code>FEATURE_DIM=23</code>
Template 框架	<code>app/domain/templates/base.py</code>	<code>TopologyTemplate</code> , <code>ParametricInvariant</code>
Template 匹配器	<code>app/domain/templates/matcher.py</code>	<code>match_template</code> , <code>match_all_templates</code>
Template 注册表	<code>app/domain/templates/registry/*.py</code>	6 类拓扑模板
Phase E 对齐	<code>app/domain/gnn/alignment_fuzzy.py</code>	<code>align_components_by_signature</code>
Wire-Aware Propagation	<code>app/domain/compare/role_propagation.py</code>	<code>propagate_canonical_from_alignment</code>
PCM Context Pack	<code>app/agent/context_pack.py</code>	<code>build_context_pack</code> , <code>classify_error_family</code>
Intent-Unified Graph	<code>app/agent/graph.py</code>	<code>run_diagnostic_graph</code> (alias <code>run_agent_graph</code>)
Intent LLM 分类	<code>app/agent/intent_llm.py</code>	<code>classify_intent_smart</code>
Semantic Router	<code>app/agent/router.py</code>	<code>SemanticRouter</code>
Datasheet KB	<code>app/services/datasheet_kb_service.py</code>	<code>DatasheetKbService</code> (hybrid)
Teaching KB	<code>app/services/teaching_kb_service.py</code>	<code>TeachingKbService</code>
Agent 主服务	<code>app/services/agent_service.py</code>	<code>AgentService</code>
API 路由	<code>app/api/v1/topology.py</code> , <code>app/api/v1/angnt.py</code>	<code>/topology/suggest</code> , <code>/angnt/ask</code>
前端拓扑推荐	<code>src/components/TopologySuggestionBanner.tsx</code>	<code>confidence_band UI</code>
前端 Agent 对话	<code>src/components/AgentChat.tsx</code>	<code>boot-ready 对话框</code>

(报告正文完)